

SQL CASE Statement — Reference Guide

A practical reference with database examples, explanations, outputs, and common use cases.

1. What is the CASE Statement?

CASE is a conditional expression in SQL. It works like an **if-else** / **if-elif-else** structure: SQL checks conditions and returns a value for the first matching condition.

Key idea: CASE is an expression, so it can be used inside SELECT, ORDER BY, GROUP BY, UPDATE, and other SQL statements.

2. Basic Syntax

```
CASE
  WHEN condition1 THEN result1
  WHEN condition2 THEN result2
  ELSE default_result
END
```

3. Simple CASE vs Searched CASE

Type	How it works	Example
Simple CASE	Compares one expression with values.	CASE status WHEN 'A' THEN 'Active' ... END
Searched CASE	Tests separate conditions.	CASE WHEN salary >= 50000 THEN 'High' ... END

4. Database Example

Create a sample employee table:

```
CREATE TABLE employees (
  employee_id INT PRIMARY KEY,
  name VARCHAR(50),
  department VARCHAR(30),
  salary INT,
  experience INT
);
```

Insert sample data:

```
INSERT INTO employees VALUES
(1, 'Aarav', 'IT', 65000, 4),
(2, 'Meera', 'HR', 42000, 2),
(3, 'Rohan', 'IT', 90000, 7),
(4, 'Isha', 'Finance', 55000, 5),
(5, 'Kabir', 'HR', 30000, 1);
```

5. Searched CASE — Classifying Salary

```
SELECT name, salary,
CASE
  WHEN salary >= 80000 THEN 'High'
  WHEN salary >= 50000 THEN 'Medium'
  ELSE 'Low'
END AS salary_level
FROM employees;
```

name	salary	salary_level
Aarav	65000	Medium

Meera	42000	Low
Rohan	90000	High
Isha	55000	Medium
Kabir	30000	Low

6. Simple CASE — Department Labels

```

SELECT name, department,
       CASE department
         WHEN 'IT' THEN 'Technology'
         WHEN 'HR' THEN 'Human Resources'
         WHEN 'Finance' THEN 'Financial Operations'
         ELSE 'Other'
       END AS department_name
FROM employees;

```

7. CASE with UPDATE

CASE can calculate a value during UPDATE. Here, the salary increment depends on experience.

```
UPDATE employees
SET salary = salary +
CASE
    WHEN experience >= 7 THEN 10000
    WHEN experience >= 4 THEN 5000
    ELSE 2000
END;
```

Warning: Always use UPDATE carefully. Add a WHERE clause when you intend to modify only selected rows.

8. CASE with ORDER BY

CASE can create a custom sorting order.

```
SELECT name, department
FROM employees
ORDER BY CASE department
    WHEN 'IT' THEN 1
    WHEN 'Finance' THEN 2
    WHEN 'HR' THEN 3
    ELSE 4
END;
```

9. CASE with Aggregation

A common pattern is conditional aggregation. The following counts employees above and below a salary threshold.

```
SELECT
    SUM(CASE WHEN salary >= 50000 THEN 1 ELSE 0 END) AS employees_50k_plus,
    SUM(CASE WHEN salary < 50000 THEN 1 ELSE 0 END) AS employees_below_50k
FROM employees;
```

10. Handling NULL Values

Use **IS NULL** or **IS NOT NULL** for NULL checks. Do not use **= NULL**.

```
SELECT name,
CASE
    WHEN department IS NULL THEN 'Not Assigned'
    ELSE department
END AS department_status
FROM employees;
```

11. Important Rules & Tips

- **END is mandatory:** Every CASE expression must finish with END.
- **ELSE is optional:** Without ELSE, an unmatched CASE returns NULL.
- **First match wins:** Conditions are evaluated from top to bottom.
- **Order matters:** Put specific conditions before broader ones.
- **Aliases help:** Use AS to name calculated columns.
- **NULL:** Use IS NULL / IS NOT NULL.

12. Quick Exam Example

Classify employees according to their years of experience:

```
SELECT name, experience,  
CASE  
    WHEN experience < 2 THEN 'Fresher'  
    WHEN experience BETWEEN 2 AND 5 THEN 'Intermediate'  
    ELSE 'Experienced'  
END AS experience_level  
FROM employees;
```

13. Memory Trick

CASE → WHEN → THEN → ELSE → END

Think: “**CASE** checks **WHEN**, gives **THEN**; otherwise **ELSE**; finally **END**.”

Note: CASE is part of standard SQL and is broadly supported, although other SQL features can vary between MySQL, PostgreSQL, SQL Server, Oracle, and other DBMSs.